Design note: state space and the Kalman filter

Status: DRAFT for review. Every DECISION box below is a recommendation awaiting Mico's sign-off; the mathematics in the appendix is to be reviewed equation by equation before any code. Companions: DESIGN_VCE.md, DESIGN_TSINFER.md. Primary references: Durbin & Koopman, *Time Series Analysis by State Space Methods* (2nd ed., 2012) — "DK" below; Koopman (1997); Koopman & Durbin (2000, 2003).

0. What this tier buys

One engine (the Kalman filter/smoother with exact diffuse initialization and ML estimation) and four front-ends in order of delivery: `ucm` (unobserved components), exact-ML `arima`, `dfactor` (dynamic factors), and a subset of `sspace`. The engine also opens the staged doors from earlier notes: multivariate lpirf and vec are unrelated, but forecasting with ragged edges, mixed-frequency nowcasting, and EM estimation all become incremental once this exists.

1. Canonical form

The internal representation is DK's:

```
y_t = Z alpha_t + eps_t,        eps_t ~ N(0, H)
alpha_{t+1} = T alpha_t + R eta_t,   eta_t ~ N(0, Q)
```

y_t is p x 1, alpha_t is m x 1, R is m x r. Time-invariant Z, H, T, R, Q only in this tier. H is DIAGONAL (required by univariate filtering, §3; every planned front-end satisfies it). No correlation between eps and eta (rarely used; a documented non-goal).

DECISION 1 (recommended): DK form; time-invariant; H diagonal; uncorrelated disturbances.

2. Initialization

Per-state classification into stationary and diffuse blocks:

- Stationary block: a_1 from the unconditional mean (0 for mean- deleted states), P_star_1 from the Lyapunov equation vec(P) = (I - T (x) T)^{-1} vec(R Q R') restricted to the block.
- Diffuse block: EXACT diffuse initialization (Koopman 1997): P_1 = P_star_1 + kappa P_inf_1 with kappa -> infinity handled analytically by carrying P_star and P_inf separately through the first d observations until rank(P_inf) collapses to zero.

The large-kappa approximation is REJECTED on numerical-constitution grounds: it manufactures O(kappa) cancellation, which is exactly the arithmetic that diverges across BLAS implementations and would break the byte-identity promise. Exact diffuse is more code and cleaner numbers, and it is what Stata and statsmodels compute, so verification stays exact.

DECISION 2 (recommended): exact diffuse for nonstationary states; Lyapunov unconditional moments for stationary states; no kappa.

3. Filter mechanics

UNIVARIATE (sequential) processing, DK §6.4: each element of y_t is treated as a scalar observation updated in turn. Consequences, all favorable:

- No matrix inversion or Cholesky in the update loop — F_{t,i} is a scalar. Best possible determinism story across BLAS backends.

- Partially missing observation vectors are handled by simply skipping the missing elements — no case analysis, no reindexing. (The WEO panels are full of ragged edges; this is the everyday case, not the corner.)
- Composes cleanly with exact diffuse (Koopman & Durbin 2000): the diffuse recursions are also scalar.

Cost: H must be diagonal (accepted in §1).

DECISION 3 (recommended): univariate filtering throughout; no multivariate covariance filter, no square-root filter.

4. Missing data and the sample contract

Fully missing y_t: prediction step only. Partially missing y_t: update on the observed elements (free under §3). The ts-tier contiguity guard is RELAXED for this tier: the time index must be gap-free after tsset, but missing VALUES inside the range are legal and handled by the filter — that is half the point of the Kalman filter. The estimation output reports the effective number of observations (scalar updates performed).

DECISION 4 (recommended): as above.

5. Smoother

DK backward recursions (§4.4): smoothed states alpha_hat_t and variances V_t via (r, N); disturbance smoother for eps_hat, eta_hat and the auxiliary (standardized) residuals. Diffuse-period smoothing via the (r0, r1, N0, N1, N2) extension (DK §5.3), matching KFAS/statsmodels. This buys `predict, smstates` / `predict, rstates` / auxiliary-residual diagnostics, and an EM option later, for one backward pass.

DECISION 5 (recommended): full DK smoother incl. disturbance smoother, from day one.

6. Likelihood and estimation

Prediction-error decomposition in scalar form: each non-diffuse scalar update contributes -0.5 (log 2 pi + log F + v^2 / F); each diffuse step with F_inf > 0 contributes -0.5 (log 2 pi + log F_inf) (Koopman 1997's diffuse likelihood — Stata's and statsmodels' convention; the number of such terms is d, reported).

Optimizer: GSL BFGS2 on the transformed parameters (§7), gradients by central differences (fixed h = 1e-5 relative), deterministic line-search constants, convergence when the relative log-likelihood improvement < 1e-9 or 200 iterations. Byte-identity stance, stated plainly: an iterative optimizer is the first place where backend rounding can change the PATH, not just the seventh digit. Defenses: (a) tight convergence so all backends land within tolerance of the same optimum; (b) regression-test goldens print estimates through round() at 1e-5 rather than raw display precision; (c) a COMPATIBILITY.md paragraph documenting the stance. If a divergence survives (a)-(b) on the three rigs, the release gate fails and we tighten — the promise outranks the feature.

DECISION 6 (recommended): as above.

7. Parameterization

- Variances: sigma^2 = exp(2 psi) (log-sigma unconstrained).
- Covariance blocks (dfactor, later): Cholesky factors, diagonal log-transformed.

- AR/MA polynomials in the arima front-end: Monahan's partial- autocorrelation transform, enforcing stationarity/invertibility during optimization (Stata's arima does the same).
- Generic sspace-subset: coefficients unconstrained (Stata's behavior); a nonstationary T during evaluation is an error with a clear message, not a crash.

DECISION 7 (recommended): as above.

8. Command surface and delivery order

Fixed-structure front-ends first; generic sspace last. The 5% argument: `ucm gdp, model(lltrend)` is typed a hundred times for every hand-written state equation.

1. `ucm depvar [if] [in], model(MODEL) [seasonal(#)]` MODELs in this release: ntrend (level only = white noise around const), llevel (local level), lltrend (local linear trend), rwalk, rwdrift. seasonal(#): trigonometric… no — DUMMY seasonal (sum-to-zero), one variance. Cycle: STAGED.
2. `arima depvar, arima(p,d,q) [sarima(P,D,Q,s)]` re-grounded on the engine: exact ML via the state-space form, replacing the current arima.c likelihood. The old command surface is preserved; the numbers move to exact ML and the change is documented loudly (this WILL change existing arima goldens — it is a correction, like Bug 39).
3. `dfactor`: one common factor, AR(1) factor dynamics, idiosyncratic AR(0) errors first (the nowcasting workhorse).
4. `sspace` subset: state equations with fixed coefficient matrices entered via a constrained syntax to be specified in its own addendum once 1-3 are shipped and the engine is trusted.

DECISION 8 (recommended): order as above; ucm's five models + dummy seasonal define release one of the tier.

9. Standard errors

Default: observed information matrix (OIM) by numerical differentiation of the score at the optimum (Stata's sspace/ucm default). `vce(robust)`: OPG-Hessian sandwich through the existing vce module (its third act). `vce(oim)` and `vce(opg)` selectable.

DECISION 9 (recommended): OIM default, robust via vce module.

10. Verification protocol

- Nile local level (DK's running example): sigma2_eps = 15099, sigma2_eta = 1469.1, and the book's filtered/smoothed trajectories. The Nile series (100 obs) is EMBEDDED as `sysuse nile` so the literature's own numbers become a regression test.
- Airline arima(0,1,1)(0,1,1)_12 exact ML against Stata's arima.
- statsmodels: element-by-element comparison of filtered and smoothed states/variances and the diffuse log-likelihood on both problems, plus a partially-missing multivariate problem for the univariate filter path.
- tools/stata_check_sspace.do mirrors the regression tests for the IMF Stata run.

Appendix A: the recursions to be implemented

Notation: for observation t, scalar element i with row Z_i of Z and observation variance h_i (diagonal of H). State prediction a, P (P = P_star during and after the diffuse phase; P_inf additionally during it).

A.1 Univariate update, standard phase (P_inf = 0), skipping missing y_{t,i}:

```
v   = y_{t,i} - Z_i a
F   = Z_i P Z_i' + h_i
K   = P Z_i' / F
a  <- a + K v
P  <- P - F K K'
ll += -0.5 (log 2 pi + log F + v^2 / F)
```

A.2 Univariate update, diffuse phase (P_inf != 0):

```
v     = y_{t,i} - Z_i a
F_inf = Z_i P_inf Z_i'
F_st  = Z_i P_star Z_i' + h_i
M_inf = P_inf Z_i'
M_st  = P_star Z_i'
```

If F_inf > 0: K0 = M_inf / F_inf a <- a + K0 v P_star <- P_star + K0 K0’ F_st - K0 M_st’ - M_st K0’ P_inf <- P_inf - K0 M_inf’ ll += -0.5 (log 2 pi + log F_inf) Else (F_inf = 0): exactly A.1 with F = F_st.

The diffuse phase ends at the first (t, i) after which P_inf = 0 (numerically: max |P_inf| < tol_inf, tol_inf = 1e-8 relative to its initial scale); d = number of scalar steps with F_inf > 0.

A.3 Transition (once per t, after the last element):

```
a <- T a
P_star <- T P_star T' + R Q R'
P_inf  <- T P_inf T'          (diffuse phase only)
```

A.4 Backward smoothing, standard phase (DK 4.4, univariate form, elements in reverse order; L = I - K Z_i):

```
r <- Z_i' v / F + L' r
N <- Z_i' Z_i / F + L' N L
```

and per time step, before stepping to t-1: r <- T’ r , N <- T’ N T Smoothed: alpha_hat_t = a_t + P_t r_{t-1}, V_t = P_t - P_t N_{t-1} P_t.

A.5 Diffuse-period smoothing: the (r0, r1, N0, N1, N2) recursions of DK §5.3 / Koopman & Durbin (2003), implemented to numerical equality with statsmodels’ `KalmanSmoother` on the verification problems. (Spelled out in the implementation addendum rather than here; the sign conventions differ across editions and the addendum will state the exact update lines being coded, for review.)

A.6 Disturbance smoother (standard phase):

```
eps_hat_{t,i} = h_i (v / F - K' r_t)
eta_hat_t     = Q R' r_t
```

A.7 Lyapunov initialization for the stationary block: solve (I - T_s (x) T_s) vec(P) = vec(R Q R’)_s by dense LU on the m_s^2 system (m_s <= ~20 for every planned front-end; cost irrelevant).

Appendix B: engine architecture

```
src/kalman.c   the engine: no Stata parsing, no printing.
    SSModel   { m, p, r; Z, Hdiag, T, R, Q; a1; Pstar1, Pinf1 }
    ss_filter(model, y, T_obs, out F/v/K per step | ll, d)
    ss_smooth(...)            (states, variances, disturbances)
    ss_loglik(model, y)       (thin wrapper used by the MLE)
    ss_mle(model-template, y, theta-map, options)
src/sspace.c   the front-ends: ucm, arima bridge, dfactor,
```

```
sspace-subset; each builds an SSModel + theta-map
and calls the engine; all printing lives here.
```

Regression testing stays command-level (house style); the engine gets its coverage through the front-ends plus the Nile/airline goldens.

Appendix C: explicitly staged out

Time-varying system matrices, correlated disturbances, non-diagonal H, square-root filtering, simulation smoother, EM estimation, mixed-frequency observation equations, `ucm` cycles, multivariate dfactor generalizations, full sspace syntax. Each is an extension of this design, not a revision of it.

Addendum A (v1.6.40): the sspace subset — syntax and semantics

Promised in §8.4; implementable now that the engine (v1.6.36-38) and the constraints subsystem (v1.6.39) exist. Stata's sspace is unusable without constraints, which is why this lands last.

Syntax

```
sspace (sname L.s1 [L.s2 ...], state [noerror])
       [more state equations]
       (depvar s1 [s2 ...] [, noerror noconstant])
       [more observation equations]
       [, constraints(numlist) covstate(identity|diagonal)
          covobs(diagonal) smstates(stub)]
```

- State equations (marked `state`): each names a NEW state and lists the LAG-1 states it loads on; every listed `L.sj` coefficient is a free parameter named [sname]L.sj. Higher lags are expressed the standard way, with auxiliary identity states (s2lag = L.s2), kept out of the syntax. No constants in state equations (staged). `noerror` drops the equation's disturbance.
- Observation equations: depvar loads on CONTEMPORANEOUS states; coefficients named [depvar]sj; constant unless noconstant; exog staged. `noerror` makes the equation an identity up to the states (H row = 0 — legal in the univariate filter).
- covstate(identity) is the default (Stata's): state disturbance variances fixed at 1, scale identified through coefficients. covstate(diagonal) frees them (log-sigma). covobs(diagonal) is the only observation form (the engine's H-diagonal requirement, §1).
- Identification is the user's job via constraints(), as in Stata. Everything the constraint language can say about the coefficient parameters is allowed; variance parameters are not constrainable (v1.6.39 policy).

Semantics and limits

- Time-invariant, stationary models only: at each likelihood evaluation T is assembled from the state-equation coefficients and P1 solved by Lyapunov; a non-PD solution (unit or explosive roots) is an evaluation error (the v1.6.39 guard). Stata's sspace has the same stationarity default; the diffuse option is staged.
- Estimation, starts, OIM, output conventions, smstates(): the dfactor machinery verbatim (BFGS2, central differences, three deterministic starts, Hessian in the reparameterized psi space).

- Verification protocol: (1) internal exactness — the AR(1) written as sspace must reproduce `arima, arima(1 0 0) noconstant` to reported precision (same likelihood function reached through a different front door); (2) a noisy-AR(1) (signal extraction) model against a hand-written statsmodels custom MLEModel with identical structure; (3) constrained-coefficient printing.